PRD — Anti-Fuckup Agent

A living blast map for AI code edits

Version 1.7 · Status *Approved for build* · Track 02 · Build with Graph Intelligence · Build window 3 hours · 3 engineers · code freeze T+2:45

> **Changes in v1.7.** One clarification, no spec change to `afu` itself. This PRD now states as a committed part of the plan — not a roadmap suggestion — that the two decision points sitting just outside `afu`'s deterministic core will be built as **dedicated agent classifiers**: an intent-normalization classifier (free text → `{symbol, change_class}`) and a scope-decision classifier (proceed / narrow / escalate, from `radius` + resolution counts + `--history`). Both consume `afu`'s JSON output; neither runs inside `afu`, and neither changes what `afu` computes or verifies. §1 names this up front; Appendix E.4 is rewritten from "hooks for the next step" to the actual classifier specs — inputs, outputs, and the decision boundary each one owns.
>
> **Changes in v1.6 — stitching release.** No new features. Every feature is now wired to the data it consumes and the state it needs, so the document can be handed to the agent-tooling step (classifiers, workflows, trees) without gaps. Specifically: (1) `public_exports` is now a list emitted by F1 and carried in the contract, so F8 has something to iterate. (2) Doc candidates get their own IDs (`d1…dN`) with an optional `node_id`, since node IDs do not cover out-of-scope exports. (3) F8 persistence is specified: `--suggest` writes `.afu/suggestions.json`, `--apply` writes `.afu/docs-state.json`; both are labelled metadata, never ground truth. (4) Doc detection runs inside `check` final mode and prints one line; `check` caches `.afu/last-diff.json`, which standalone `afu docs` reuses — this is how the "no duplicate diff" criterion is actually met. (5) The doc-map schema is defined, keyed on `file#symbol`. (6) `--base`/`--head` default from the contract. (7) `--apply` loses `--json` and its undefined exit 1. (8) The model client lives in `suggest.py` behind a lazy import and an optional extra, so plain `afu` never imports it. (9) Egress is now honesty rule 8. (10) One shared `change_class` enum is used by `brief` (intended), `check` (observed), and `docs` (doc-affecting) — §5 F2. (11) New §6.5 lists every file under `.afu/` with its writer, readers, and trust level. (12) New Appendix E is the agent integration contract. (13) G9 is marked as shipping with F8. (14) F8's draft layer is explicitly post-freeze; only the detection layer is in the 3-hour stretch. (15) Appendix A table repaired. (16) §4 diagram restored. Sections that v1.5 referenced as "unchanged" are written out in full so the document stands alone.
>
> **v1.5.** `afu docs --suggest` / `--apply`: the one model-assisted, human-confirmed doc draft. **v1.4.** F8 `afu docs` detection layer promoted from roadmap. **v1.3.** One graph, four moments; `map`, `shadow`, three-state resolution, `--radius`, `--history`. **v1.2.** Databricks lane removed. **v1.1.** Bound to the published `entire-graph` command surface.

---

1. Summary

An AI agent editing a codebase does not know what it does not know. It changes a shared function, misses four consumers, and the build breaks. The failure is not that the model wrote bad code — the model wrote reasonable code against an incomplete picture.

Anti-Fuckup Agent (`afu`) gives the agent the picture before it edits, keeps the picture alive while it edits, and proves afterwards that it stayed inside the picture. Every other agent tool shows a diff after the fact. `afu` shows the blast zone before the first keystroke, marks where its own analysis gives up, proves the claim with a counterfactual run, and names the docs that now lie about the interface.

It wraps `entire-graph`, the sponsor's open-source code knowledge graph, and adds what the raw graph does not do: a scoped, ordered, agent-readable **map**; a **contract** derived from that map; a mechanical **check** that fails loudly; a **shadow run** that shows what the same agent would have broken without it; and a **doc flag** on the report.

Positioning statement. Not bug-free code. An agent that cannot fuck up quietly — blast radius known before the edit, every unresolved region labelled rather than hidden, scope verified after, the counterfactual on screen, stale docs named rather than left to rot. *Fix the most painful mistakes before they ever pop.*

Why it wins. Honesty is the moat. Competitors cannot copy "we tell you exactly where we're wrong" without admitting they were hiding it.

Determinism. Every command is offline, deterministic, and model-free, with exactly one named exception: `afu docs --suggest` may call a model to draft doc prose for a human to accept, edit, or reject. It has no path into the contract, the state file, or any verdict.

Classifiers. `afu` computes and verifies; it does not decide. Two decision points sit immediately outside it, and both **will be built as agent classifiers** consuming `afu`'s JSON, not folded into `afu` itself: an **intent-normalization classifier** that turns a developer's or an agent's free-text task into the `{symbol, change_class}` pair `afu brief` / `map` require, and a **scope-decision classifier** that reads a rendered map's radius, resolution counts, and `--history` to decide proceed / narrow scope / escalate to a human, before a single edit is made. Keeping these as separate classifiers rather than model calls inside `afu` is what lets `afu` stay the deterministic, offline, model-free contract layer while the agent side still gets a real decision-making step. Full specs — inputs, outputs, decision boundaries — are in Appendix E.4.

---

2. Problem

2.1 Primary user

Priya, a developer six months into a 12k-line React/TypeScript codebase she did not write. She works through an agent. She can read a diff; she cannot read a dependency graph, and neither can the agent. What she needs is not more capability but a shared, honest picture of what a change will touch, before it touches it. (A senior developer on an unfamiliar repo has the identical problem; "junior" describes the situation, not the seniority.)

Secondary user: the agent, as a machine consumer of the map, the plan, and the contract. Anything Priya sees in a terminal, the agent sees as JSON from the same query, with the same node IDs. Appendix E is written for it.

2.2 What is difficult today

1. **The agent edits blind.** It greps, finds three usages, misses the fourth behind a barrel export or a context provider, and changes the signature anyway. → *F1* `afu map`
2. **A diff shows what changed, never who depended on it.** Review catches style, not reachability. → `--radius`, *F4* `afu check`
3. **Correct refactors still break the build mid-flight.** The old contract is removed before the last consumer migrates. The failure is sequencing, not logic. → *F3* `afu plan`
4. **Nobody can see where the analysis gives up.** Static resolution is unsound: call-graph studies report roughly 11% of reachable methods missed in Java and around 60% of JS call sites unresolved by precision-tuned tooling (Appendix D). Today those gaps are invisible. → *three-state resolution, the* `unknown` *bucket*
5. **Tools that claim safety are worse than tools that say nothing.** One confident "nothing else affected" that turns out wrong costs trust permanently. → *§3.3 honesty rules*
6. **Value is asserted, never shown.** Every guardrail tool says it prevents breakage. None shows you the run it prevented. → *F5* `afu shadow`

7. **A fix that changes the interface silently breaks the docs.** A bug fix changes a signature, a default, or an error message the user relies on. Nobody updates the docs; nothing catches it. Same blind spot as 1 and 2, pointed at prose. → *F8* `afu docs`

2.3 Why now

The sponsor ships a fast, local, deterministic code graph. The primitive exists. What does not exist is the decision layer that turns graph facts into an agent workflow with a verifiable contract and a visible counterfactual.

Alignment note. The sponsor's stated design position is that a wrong edge is worse than a missing one, and they publish accuracy as the product — 94% versus 68% for the leading tree-sitter code-memory tool on a frozen 21-repo board. `afu`'s three-state resolution and unknown bucket are that philosophy applied one layer up: a confident wrong verdict is worse than an admitted gap. We are extending their position, not wrapping their binary.

---

3. Goals

3.1 Goals

#	Goal	Measured by	Ships with
G1	Agent receives structural context before writing	Brief + map generated and injected in < 3s	F1 (P0)
G2	Multi-step changes never leave the build red	Guided run: 0 broken intermediate states	F3 (P0)
G3	Out-of-scope edits are caught mechanically	`afu check` exits non-zero on violation	F4 (P0)
G4	Unresolvable regions are surfaced, never hidden	Every output includes an `unknown` bucket; every non-sound node carries a reason	F1/F2 (P0)
G5	Value is proven per run, not per demo	`afu shadow` prints naive-vs-guided breakage on every run	F5 (P1)
G6	Agent and human see the affected subgraph before any edit	`afu map` renders the rooted tree in < 3s on the demo repo	F1 (P0)
G7	The map *is* the scope contract	100% of files edited in a guided run appear in the approved map, or `afu check` fails	F1/F4 (P0)
G8	Uncertainty is spatial, not a footnote	Unknown nodes appear in the tree where resolution failed	F1 (P0)
G9	Interface-facing changes never ship without a doc flag	Every signature-class change to a public export produces one doc-candidate line, `sound` or `unknown`	F8 detection (P2 stretch)

3.2 Non-goals

- Not claiming bug-free or bug-free-adjacent output. Undecidable, and the underlying resolution is unsound by construction.
- Not rebuilding the graph. `entire-graph` is the substrate; we are a consumer.
- Not a UI, IDE extension, or web dashboard. Terminal and JSON only. The map is ANSI, not a canvas.
- Not multi-language on day one. One TypeScript/React repo.
- Not semantic understanding of behaviour, **with one scoped exception:** `afu docs --suggest` may call a model to draft replacement doc *prose* for an already-flagged candidate. Every other command reasons about structure and change class only and never calls a model.
- Not driving the agent, **with the same exception.** `afu` informs, gates, and verifies. It never calls a model to decide what to edit, when, or whether a violation is real. The one model call drafts text for a human; it has no path into the agent's edit loop, the contract, or `check`'s verdict.
- **Not auto-patching docs without a human in the loop.** `--suggest` drafts; `--apply` writes one reviewed suggestion, one file, one confirmation. There is no accept-all.

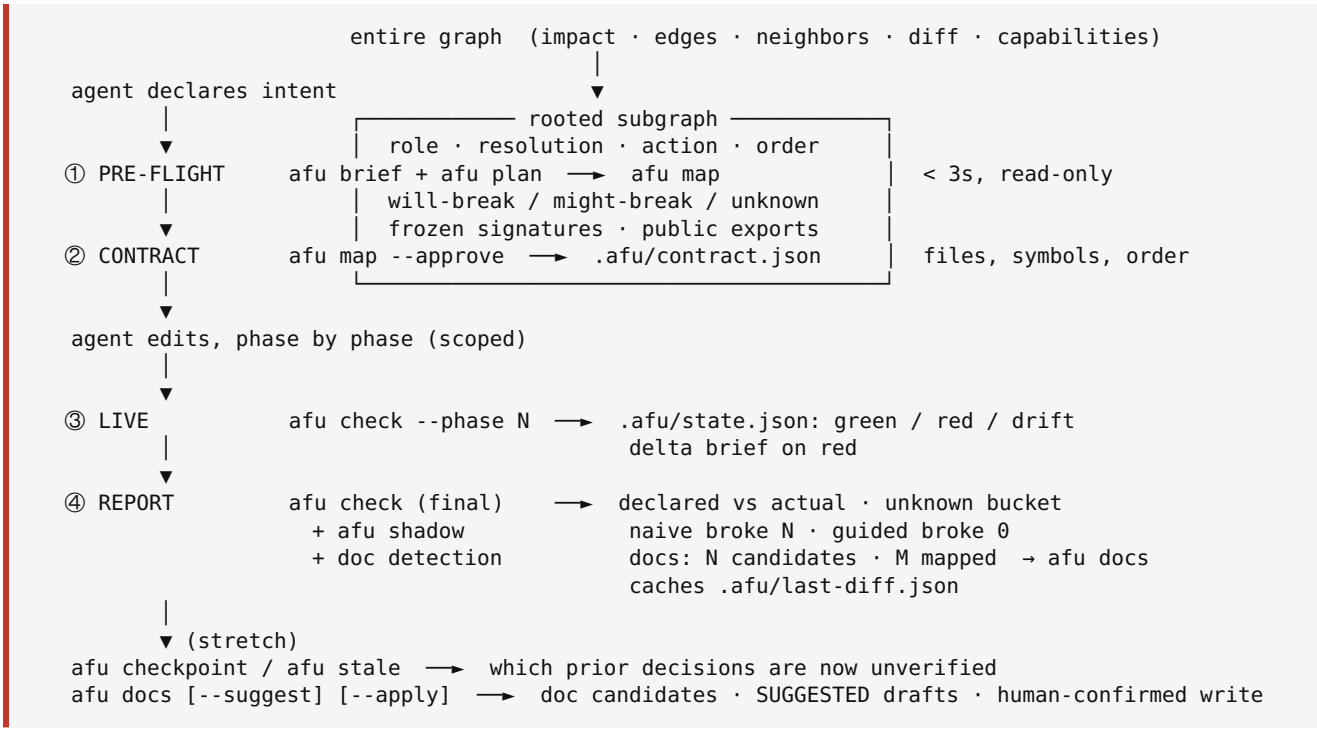
3.3 Honesty rules — explicit anti-requirements

Product requirements, not caveats. Violating one is a P0 bug.

1. The tool never prints a verdict word — *safe, clean, verified, OK* — as a summary. It prints counts and the bucket.
2. The resolution line always prints, including when unknown = 0, with the qualifier that makes it true: "*0 unknown among static imports; 4 dynamic imports unchecked.*"
3. No node is shown as `sound` without a resolved graph edge. Name matches and co-change are `guessed`, full stop.
4. Every `guessed` and `unknown` node carries a one-line reason naming the rule or the gap.
5. `afu shadow` reports what the *guided* run broke too. If it is not zero, it says so.
6. `afu docs` never claims a doc is up to date — only that no flagged change touched it. A doc outside doc-map coverage is `unmapped`, never silently clean.
7. A model-drafted suggestion is always labelled `SUGGESTED —` in every rendering, human and JSON. It is stored only in `.afu/suggestions.json`, never in the contract, the state file, or any file `afu` treats as ground truth (§6.5). `--apply` never fires without an interactive per-file confirmation in the same process; no flag, env var, or config makes it non-interactive.
8. **Egress is named.** The graph runs with `no_egress=true` (verified at T+0:20). The only network call in the tool is `afu docs --suggest`. It is off unless the flag is passed; it sends exactly the old signature, the new signature, and the current doc paragraph; `afu doctor` prints whether a model credential is present and states that `--suggest` is the only command that would use it.

---

4. Solution overview — one graph, four moments



Doc detection is not a fifth moment: it is a fourth view on the `DiffResult` that `check` already holds at REPORT time. Same node IDs at all moments; the agent references `n7` in its plan and `check` matches it to a diff hunk. Everything is a subprocess call to `entire graph` plus parsing plus one graph algorithm. **No model calls inside `afu`, except `docs --suggest`.**

---

5. Feature specifications

F1 · `afu brief` + `afu map` — pre-edit impact brief and rendered blast map · P0

```
afu brief --symbol useLikedSongs --depth 2 --change signature [--json] [--repo .]
afu map   --symbol useLikedSongs --depth 2 --change signature [--radius] [--history] [--approve]
[--json]
```

User story. As an agent about to modify a symbol, I need every consumer, which will break, how sure the graph is about each, which signatures I am forbidden to change, and in what order to move — before I read a single file.

Design. `brief` is the data; `map` is the view. `map` = `brief` ∪ `plan`, rendered as a tree rooted at the target symbol. No third data source. `--change` names the intended change class (F2 enum); it drives the `will_break` rule and defaults to `signature`.

Behaviour (`brief`).

1. Shell out to `entire graph impact --repo <r> --symbol <s> --depth <d> --profile full`. Parse text (§6.4 note 1) unless the T+0:20 gate proves `--format json` works.
2. Parse callers, callees, type consumers, data flows, co-change files, related files.
3. Classify every consumer into exactly one **risk bucket** and stamp exactly one **resolution state** (F2).
4. Derive `frozen_signatures`: the exported signature of the target and of every direct caller that destructures or positionally consumes it.

5. Derive `public_exports`: every exported symbol in `files_in_scope`, as `file#symbol`. **This list is what F8 iterates.**
6. Derive `tests_in_scope`: test symbols that reach the target.
7. Seed the unknown bucket from `entire graph capabilities --json`. Anything out of coverage is `unknown` by construction.
8. Assign stable node IDs `n1…nN` in risk order (F2 ordering rule).

Behaviour (`map`).

1. Run `brief`, then `plan` (F3), on the same subgraph.
2. Render each node as one line: **id · path · role · resolution · order · action**, plus an indented reason line for every non-sound node.
3. Sort within a level: `unknown` first, then `guessed`, then `sound`.
4. Collapse siblings under one directory past 5 as `dir/** (N files)` with aggregate resolution.
5. Default depth 2. `--depth` expands.
6. `--radius` appends: files · packages crossed · public exports · resolved consumers · tests affected.
7. `--history` appends per-node repo memory: how often this file went red in past changes of the same class (git log co-change, parser-free).
8. `--approve` writes `.afu/contract.json` (§6.5) and seeds `open_questions` from the unknown bucket (F6). It is the only flag on `map` that writes.
9. `--json` emits the identical structure. The agent never parses box-drawing characters.

Human output.

```
afu map --symbol useLikedSongs --change signature --radius --history

  n1  src/hooks/useLikedSongs.ts               TARGET     ● sound    #4 edit
  ├─ n2  src/features/liked/LikedList.tsx      consumer   ● sound    #1 edit
  │        will_break: destructures array return (hop 1)
  ├─ n3  src/features/player/PlayerQueue.tsx   consumer   ● sound    #2 edit
  │        might_break: transitive via useQueue (hop 2)
  ├─ n4  src/api/songs.ts                      coupled    ◐ guessed  #3 edit, verify
  │        might_break: co-change 38/50 commits, no resolved edge · red in 2 of last 4 changes here
  ├─ n5  src/router/registry.ts                consumer?  ○ unknown  read-only
  │        unknown: string-keyed handler map, 1 dynamic dispatch site, graph cannot resolve
  └─ tests
     ├─ n6  useLikedSongs.test.ts              covers n1             #5 edit
     └─ n7  LikedList.test.tsx                 covers n2             run

  radius      5 files · 1 package · 2 public exports · 3 resolved consumers · 2 tests
  resolution  3 sound · 1 guessed · 1 unknown  (3 unresolved call sites, 1 dynamic dispatch)
  frozen      useLikedSongs, formatDuration
  exports     src/hooks/useLikedSongs.ts#useLikedSongs, src/lib/format.ts#formatDuration
  order       Phase 1 additive → n2, n3 → n4 → n1 removal (gated on 0 dependents)
```

Output contract (`--json`). This is also the contract file schema after `--approve` (§6.5 adds `approved_at` and `contract_id`).

```
{
  "symbol": "useLikedSongs", "file": "src/hooks/useLikedSongs.ts",
  "intended_change": "signature", "depth": 2,
  "generated_at": "2026-09-06T09:40:12Z", "base_sha": "a1b2c3d",
  "nodes": [
    { "id": "n2", "symbol": "LikedList", "file": "src/features/liked/LikedList.tsx",
      "role": "consumer", "hop": 1, "bucket": "will_break", "resolution": "sound",
      "edge_ids": ["e_9f2c"], "reason": "destructures array return", "action": "edit",
      "order": 1, "history": { "red_in_last": [2, 4] } },
    { "id": "n5", "file": "src/router/registry.ts", "role": "consumer?", "hop": null,
      "bucket": "unknown", "resolution": "unknown", "edge_ids": [],
      "reason": "string-keyed handler map; 1 dynamic dispatch site", "action": "read-only" }
  ],
  "unknown": { "unresolved_callsites": 3, "dynamic_dispatch_sites": 1,
               "out_of_coverage": [], "notes": ["handler map in src/router/registry.ts"] },
  "radius": { "files": 5, "packages": 1, "public_exports": 2, "consumers": 3, "tests": 2 },
  "frozen_signatures": ["src/hooks/useLikedSongs.ts#useLikedSongs", "src/lib/
format.ts#formatDuration"],
  "public_exports":    ["src/hooks/useLikedSongs.ts#useLikedSongs", "src/lib/
format.ts#formatDuration"],
  "tests_in_scope":    ["LikedList.test.tsx", "useLikedSongs.test.ts"],
  "files_in_scope":    ["src/hooks/useLikedSongs.ts", "src/features/liked/LikedList.tsx",
                        "src/features/player/PlayerQueue.tsx", "src/api/songs.ts"],
  "plan": [ { "phase": 1, "kind": "additive", "nodes": ["n1"] },
            { "phase": 2, "kind": "leaves",   "nodes": ["n2", "n3"] },
            { "phase": 3, "kind": "interior", "nodes": ["n4"] },
            { "phase": 4, "kind": "removal",  "nodes": ["n1"], "gate": "dependents == 0" } ]
}
```

Acceptance criteria.

- AC1: `brief` and `map` each return in under 3 seconds on the demo repo (snapshot-query path).
- AC2: `unknown` is always present, even when empty, with the §3.3 rule 2 qualifier.
- AC3: `--json` parses as valid JSON with no log lines on stdout.
- AC4: Exit 0 on success; exit 2 with a readable message if the symbol is not found.
- AC5: Every node carries exactly one `bucket`, one `resolution`, and — if not `sound` — a non-empty `reason`. Every `sound` node has ≥ 1 `edge_ids`.
- AC6: `map` without `--approve` performs zero writes (verified on a read-only checkout).
- AC7: Node IDs in `--json` and the ANSI tree are identical for the same invocation.
- AC8: `public_exports` and `frozen_signatures` use the `file#symbol` form everywhere.

F2 · Classification — change class × risk bucket × resolution state · P0

Three mechanical axes, each explainable in one sentence per value.

Change class — *what kind of change is this?* One enum, used three ways: `brief --change` (intended), `check` (observed, from `graph diff`), `docs` (doc-affecting subset).

change_class	Observed as (`graph diff`)	Signature-class?	Doc-affecting?
`add`	`added`	yes	yes, if public export
`remove`	`removed`	yes	yes, if public export
`rename`	`renamed`	yes	yes, if public export
`signature`	`signature-changed` (params, return shape)	yes	yes, if public export
`move`	`removed` + `added` same symbol, different file	yes (import paths)	yes, if public export
`body`	`body-changed`	no	`unknown` if public export

Risk bucket — *how likely is this consumer to break?*

Bucket	Condition
`will_break`	Direct resolved caller (hop 1) and `intended_change` is signature-class
`might_break`	Hop-2 resolved caller; or `body` change with a resolved caller; or co-change coupling ≥ 60% over last 50 commits with no resolved edge
`unknown`	Unresolved call site, dynamic dispatch, string-keyed lookup, reflection, any construct the parser reports without a resolved target, or any file outside `capabilities --json` coverage

Resolution state — *how sure is the graph that this edge exists?*

State	Glyph	Condition
`sound`	●	≥ 1 resolved `CALLS` / `IMPORTS` / type edge with an edge ID
`guessed`	◐	No resolved edge; included by co-change or name match; heuristic named in `reason`
`unknown`	○	Parser saw an unresolvable reference, or file is out of coverage

Coupling rules: `will_break` ⇒ `sound`. `unknown` bucket ⇔ `unknown` resolution. `might_break` may be `sound` (hop 2) or `guessed` (co-change).

Risk ordering within a bucket: hop ASC, then `has_test` DESC, then co-change frequency DESC. Node IDs are assigned in this order.

Acceptance criteria.

- AC1: Every consumer lands in exactly one bucket and one resolution state.
- AC2: The `reason` string names the rule that fired.
- AC3: A symbol with zero consumers and zero co-change still produces output with an explicit "graph found nothing" note.
- AC4: No node is `sound` without an edge ID (§3.3 rule 3).
- AC5: `check` maps every `graph diff` entity to exactly one `change_class`; `move` is detected as remove+add of the same `symbol` name across files.

F3 · `afu plan` — ordered refactor sequence · P0

```
afu plan --symbol useLikedSongs [--json] [--allow-cycles]
```

User story. As a developer making a breaking change across several consumers, I need an order where every intermediate state compiles and tests pass.

Behaviour.

1. Reverse-dependency subgraph from the graph directly: `entire graph edges --repo . --format ndjson --to <symbol> --relation CALLS --profile full`; fallback `entire graph neighbors --symbol <s> --relation CALLS --direction in --depth 2`.
2. Topological sort with `graphlib.TopologicalSorter` (stdlib; raises `CycleError`).

3. Four-phase plan: **Additive** → **Leaves** → **Interior** → **Removal** (gated on recomputed dependents = 0).
4. Stamp each node's `order`.
5. On `CycleError`, print cycle members, state the change cannot be performed incrementally, exit 3.

Invariant. The old contract is never removed while any consumer depends on it. Phase 4 is gated on a recomputed dependent count, not on the plan being "done".

Acceptance criteria. AC1: each phase independently green. AC2: cycles reported with member names, exit 3. AC3: steps reference node IDs, files, symbols — never generic advice. AC4: a no-op change produces a one-step plan.

F4 · `afu check` — scope contract, live and final · P0

```
afu check --contract .afu/contract.json [--phase N] [--base <sha>] [--head <sha>] [--json]
```

`--base` defaults to `contract.base_sha`; `--head` defaults to `HEAD`.

User story. As a reviewer, I need to know mechanically whether the agent stayed inside what it declared, before I read a line of the diff. As an agent mid-run, I need to know after each phase whether I am still green and still in scope.

Behaviour.

1. Run `entire graph diff --base <base> --head <head>` (alias `analyze`); single-commit fast path `entire graph commit [REV]`. Write the parsed `DiffResult` to `.afu/last-diff.json` with `base`, `head`, and a hash of both trees (§6.5).
2. Map each entity change to a `change_class` (F2) with dependent counts.
3. Match every changed entity to a node ID in the contract. Unmatched ⇒ `OUT_OF_SCOPE`.
4. Evaluate violations:

Violation	Trigger	Severity
`FROZEN_SIGNATURE`	A `frozen_signatures` entry is signature-class changed	Blocking
`OUT_OF_SCOPE`	A changed symbol is absent from the contract's node set	Blocking
`UNDECLARED_REMOVAL`	A symbol was removed that had resolved dependents	Blocking
`PHASE_DRIFT`	`--phase N` given and a node scheduled for a later phase was changed	Blocking
`UNGUARDED_CHANGE`	A changed symbol is reached by zero tests	Warning
`UNKNOWN_TOUCHED`	A file in the `unknown` bucket was changed	Warning — escalate to human

1. **Live mode (`--phase N`).** Update `.afu/state.json` with per-node `green / red / drift`. On red, emit a **delta brief**: the failing node, its consumers, its reason — same shape as pre-flight, not a compiler dump.
2. **Final mode (no `--phase`).** Print the report: declared vs actual footprint, verified-green nodes, unknown bucket, the `shadow` line if `.afu/shadow.json` exists, and **the doc line** — F8 detection run in-process on the same `DiffResult`: `docs 2 candidates · 1 mapped → afu docs`. Detection never changes `check`'s exit code.

3. Exit accordingly.

Exit codes. 0 no blocking violation · 1 blocking violation · 2 bad input · 3 graph error.

Output.

```
VIOLATION  FROZEN_SIGNATURE
  formatDuration  src/lib/format.ts
  signature changed; 7 resolved consumers; not in contract

WARNING    UNKNOWN_TOUCHED
  src/router/registry.ts  (n5)
  file was in the unknown bucket; graph cannot verify this edit — human review required

compared 12 symbols against contract c_7d1e (a1b2c3d → 9e8f7d6)
declared 5 files · actual 6 files · 1 outside contract
resolution 3 sound · 1 guessed · 1 unknown
shadow     naive broke 2 (n2, n3) · guided broke 0
docs       2 candidates · 1 mapped  → afu docs
1 blocking, 1 warning                                          exit 1
```

Acceptance criteria.

- AC1: Exit code non-zero for any blocking violation, always.
- AC2: A no-violation run prints what was checked — never a verdict word.
- AC3: Runs correctly with no model available.
- AC4: `--phase N` after a correctly executed phase N produces zero `PHASE_DRIFT`.
- AC5: Final mode writes `.afu/last-diff.json`; a following `afu docs` with the same base/head performs no second `graph diff` call (verified by subprocess log).
- AC6: The doc line prints in final mode even when candidates = 0 (`docs 0 candidates among 3 public exports changed: 0`).

F5 · `afu shadow` — the counterfactual run · P1 (demo-critical)

```
afu shadow --contract .afu/contract.json --naive <patch|sha> --guided <sha> [--build "npm run
build"] [--json]
```

User story. As a judge, a reviewer, or Priya, I want to see what the same agent on the same task would have broken without `afu`, every run, not once on a slide.

Behaviour.

1. `git worktree add /tmp/afu-shadow <contract.base_sha>`; apply the naive patch.
2. Run the build in the worktree; parse failures via `<build> 2>&1 | entire graph explain --repo /tmp/afu-shadow`; map broken entities to node IDs where the graph can.
3. Same against the guided head.
4. Write `.afu/shadow.json` (§6.5); `check` final mode prints it. Mark ✗ on the map for nodes the naive run broke.
5. Remove the worktree. Never touches the primary tree.

Acceptance criteria. AC1: primary working tree hash unchanged before and after. AC2: guided count is measured and printed as-is (§3.3 rule 5). AC3: completes in under 60s on the demo repo, or prints a partial with a timeout note.

Demo note. The naive patch is recorded at setup so Beat 1 is one command.

F6 · `afu checkpoint` / `afu stale` — verifiable agent memory · P1 (stretch)

Ledger `.afu/checkpoint.json`: pointers plus hashes, never pasted source. Node IDs are the map's `n*` IDs, plus `contract_id` so a ledger cannot be applied to the wrong contract.

- `open_questions` is seeded at `map --approve` from the unknown bucket — one question per unknown node. Escalation is a first-class output.
- `afu stale` re-diffs `contract.base_sha` against HEAD and flags decisions whose grounding hash changed. This *is* "how did the subgraph change since plan time"; no separate command.

Acceptance criteria. AC1: no source text in the ledger. AC2: `stale` flags a decision after its grounding symbol is edited. AC3: rehydration for a 40-node evidence set under 1,000 tokens. AC4: ledger carries `contract_id`; mismatch exits 2.

F7 · Test selection · P2 (stretch)

`entire graph verify --record-baseline` during setup, `--pre-edit-baseline` after the edit. Feeds the `covers nX` annotations on the map's `tests` branch and the `UNGUARDED_CHANGE` warning — one source, two views. **Safety:** `verify` executes the command you give it; only ever pass the demo repo's own test command.

F8 · `afu docs` — doc-staleness surface, with model-assisted draft · P2 (stretch); draft layer post-freeze

```
afu docs [--contract .afu/contract.json] [--base <sha>] [--head <sha>] [--doc-map .afu/doc-map.json] [--json]
afu docs ... --suggest [--json]
afu docs --apply <candidate-id>
```

Defaults: `--contract` `.afu/contract.json`; `--base` `contract.base_sha`; `--head` `HEAD`; `--doc-map` `.afu/doc-map.json` if present.

User story. As Priya, when a bug fix changes a public function's signature, a default, or an error message, I need to be told which docs might now be wrong, where to look, and — if I want it — a starting-point rewrite I can read, edit, or discard, without `afu` touching the file until I say so.

Design. Two layers; the boundary is the one place a model may run.

- **Detection layer** (`afu docs`, no flag) — mechanical, model-free, zero writes. Runs in-process inside `check` final mode (F4 step 6) and standalone.
- **Draft layer** (`--suggest`, `--apply`) — the one model call and the one write into the primary working tree outside `.afu/`. Runs only on candidates the detection layer produced *and* that have a doc-map hit.

Doc map schema (`.afu/doc-map.json`, maintainer-supplied, human-owned).

```
{
  "version": 1,
  "entries": [
    { "symbol": "src/hooks/useLikedSongs.ts#useLikedSongs",
      "doc_file": "docs/hooks/liked-songs.md", "anchor": "## Usage" },
    { "symbol": "src/lib/format.ts#formatDuration",
      "doc_file": "docs/api/format.md" }
  ]
}
```

Keyed on `file#symbol`, never on node IDs (which renumber per `--approve`). `anchor` is optional; absent means whole file.

Behaviour — detection.

1. Load `DiffResult` from `.afu/last-diff.json` if its base/head match; otherwise run `graph diff` once and write the cache.
2. For each changed entity whose `file#symbol` is in `contract.public_exports` **or** is an exported symbol per `DiffResult` (covers out-of-scope exports), classify `change_class` (F2).
3. Stamp `change_resolution`: `sound` for signature-class, `unknown` for `body`. **A doc-map hit never changes this field** (§3.3 rule 6). It grades the *change*, not the doc link.
4. Look up the doc map: hit ⇒ `doc_link: "mapped"` with `doc_files`; miss ⇒ `doc_link: "unmapped"` and the line ends "*no mapped doc — human judgement.*"
5. Assign candidate IDs `d1…dN` in order: signature-class first, then `body`; within class by dependent count DESC. Add `node_id` when the entity matches a contract node.
6. Print one line per candidate. Zero writes.

Behaviour — `--suggest`.

1. For every candidate with `doc_link: mapped`, read the current doc section (whole file, or from `anchor` to the next heading of equal or higher level).
2. Lazy-import `suggest.py` (§6.3). If the optional extra is not installed or no credential is present, mark every mapped candidate `suggestion.status: "unavailable"` with the reason, print the rest of the report unchanged, exit as plain `docs` would.
3. Call the model once per candidate with exactly three inputs, all taken from `DiffResult` or the doc file — never re-derived by the model: old signature, new signature, current doc paragraph. Prompt: "*Rewrite this paragraph to match the new signature. Do not invent behaviour not shown in the diff.*"
4. Render as an old/new diff under the candidate, every line prefixed `SUGGESTED –`.
5. Write `.afu/suggestions.json` (§6.5): `run_id`, `base`, `head`, `doc_map_hash`, candidates with `suggestion` objects and `target_hash` (sha256 of the doc section text at draft time). This file is metadata; `check`, `map`, `plan`, `stale` never read it.

Behaviour — `--apply <candidate-id>`.

1. Load `.afu/suggestions.json`. If the id is absent, or `target_hash` no longer matches the doc section, or `doc_map_hash` changed: exit 2 with "*stale — re-run afu docs --suggest*". Nothing written.
2. Re-print the suggested diff.
3. Prompt `Apply this change? [y/n/e]` on the TTY. No non-interactive path exists.
4. `e` opens the draft in `$EDITOR`; on return, re-show and prompt `Write this? [y/n]`. The model is never called again; the second confirmation is on the developer's own text.
5. On `y`, replace exactly that section of that one doc file. On `n` or anything else, exit 0, nothing written.
6. Append to `.afu/docs-state.json` (§6.5): `{ candidate, symbol, doc_file, symbol_signature_hash, doc_section_hash_after, origin: "suggested"|"edited", applied_at }`. Detection suppresses a candidate whose `symbol_signature_hash` and doc section hash both match a record here — "resolved until the signature or the doc changes again."
7. One candidate per invocation. No batching.

Output (`--suggest`).

```
afu docs --suggest

DOC-CANDIDATE [d1 · n1]  useLikedSongs             src/hooks/useLikedSongs.ts
  change: signature (sound) · doc: mapped → docs/hooks/liked-songs.md ## Usage

  SUGGESTED — docs/hooks/liked-songs.md, "Usage"
  SUGGESTED — - `useLikedSongs()` returns an array of liked song IDs.
  SUGGESTED — + `useLikedSongs()` returns `{ ids: string[], isLoading: boolean }`. Code that
  SUGGESTED — + destructures the array directly will need to read `.ids` instead.
  → afu docs --apply d1

DOC-CANDIDATE [d2]  formatDuration                  src/lib/format.ts
  change: body (unknown) · doc: unmapped — no mapped doc, human judgement
  (no suggestion drafted — nothing in doc-map to anchor it to)

2 candidates · 1 mapped · 1 drafted · 1 unmapped · 0 previously resolved
```

Output contract (`--json`).

```
{
  "run_id": "r_3a9f", "contract_id": "c_7d1e", "base_sha": "a1b2c3d", "head_sha": "9e8f7d6",
  "doc_map_hash": "sha256:4c1…",
  "candidates": [
    { "id": "d1", "node_id": "n1", "symbol": "src/hooks/useLikedSongs.ts#useLikedSongs",
      "change_class": "signature", "change_resolution": "sound",
      "doc_link": "mapped", "doc_files": ["docs/hooks/liked-songs.md"], "anchor": "## Usage",
      "suggestion": { "status": "drafted", "label": "SUGGESTED",
        "target_file": "docs/hooks/liked-songs.md", "target_section": "Usage",
        "target_hash": "sha256:9b2…",
        "old_text": "`useLikedSongs()` returns an array of liked song IDs.",
        "new_text": "`useLikedSongs()` returns `{ ids: string[], isLoading: boolean }`. Code that
destructures the array directly will need to read `.ids` instead." } },
    { "id": "d2", "node_id": null, "symbol": "src/lib/format.ts#formatDuration",
      "change_class": "body", "change_resolution": "unknown",
      "doc_link": "unmapped", "doc_files": [], "suggestion": { "status": "not_applicable" } }
  ],
  "summary": { "candidates": 2, "mapped": 1, "drafted": 1, "unmapped": 1, "resolved_suppressed":
0 }
}
```

Exit codes. `docs` / `docs --suggest`: 0 report printed · 2 bad input · 3 graph error. `docs --apply`: 0 written or declined · 2 stale or unknown id.

Acceptance criteria.

- AC1: Every candidate has exactly one `change_class`, one `change_resolution`, one `doc_link`.
- AC2: `body` is never `sound`; a doc-map hit never changes `change_resolution`.
- AC3: Unmapped candidates always print rather than error; zero doc-map entries is a valid configuration.
- AC4: Standalone `docs` after `check` final with matching base/head issues no `graph diff` subprocess.
- AC5: `docs` and `docs --suggest` write nothing outside `.afu/`; `docs` without `--suggest` writes nothing at all except `last-diff.json` on cache miss.
- AC6: Every `suggestion` object with `status: drafted` carries `label: "SUGGESTED"`, and every rendered line of drafted text starts with `SUGGESTED —`. No other code path emits drafted text.
- AC7: `--apply` has exactly one write call to a doc file, reachable only after a TTY confirmation in the same process. No `--yes`, `--force`, or env var bypass exists.
- AC8: `--apply` on an unknown id, a `target_hash` mismatch, or a `doc_map_hash` mismatch exits 2 and writes nothing.
- AC9: A failed model call degrades one candidate to `unavailable`; command exit code equals plain `docs`.

- AC10: `afu doctor` passes without the optional extra installed; it prints `model credential: present|absent (used only by docs --suggest)`.
- AC11: After `e`, the written text is the developer's; `docs-state.json` records `origin: "edited"`.
- AC12: `import afu.suggest` never executes unless `--suggest` is passed (verified by `python -X importtime`).

---

6. CLI specification

6.1 Command surface

Command	Priority	Exit codes	Writes
`afu brief --symbol S [--depth N] [--change C] [--json]`	P0	0, 2, 3	—
`afu map --symbol S [--depth N] [--change C] [--radius] [--history] [--approve] [--json]`	P0	0, 2, 3	`contract.json` (`--approve` only)
`afu plan --symbol S [--json] [--allow-cycles]`	P0	0, 2, 3	—
`afu check [--contract F] [--phase N] [--base] [--head] [--json]`	P0	0, 1, 2, 3	`state.json` (`--phase`), `last-diff.json`
`afu shadow --naive P --guided S [--build CMD] [--json]`	P1	0, 2, 3	`shadow.json`; own worktree only
`afu checkpoint write` · `afu stale`	P1	0, 1, 2	`checkpoint.json` (`write` only)
`afu docs [--contract] [--base] [--head] [--doc-map] [--suggest] [--json]`	P2	0, 2, 3	`last-diff.json` (miss), `suggestions.json` (`--suggest`)
`afu docs --apply <id>`	P2 · post-freeze	0, 2	one doc file + `docs-state.json`
`afu doctor`	P0	0, 3	—

6.2 Global conventions

- `--repo` defaults to `.`; `--json` suppresses all decorative output. JSON to stdout, diagnostics to stderr, always.
- `--contract` defaults to `.afu/contract.json`; `--base` defaults to `contract.base_sha`; `--head` defaults to `HEAD`.
- Every command is read-only except as listed in §6.1 "Writes". `docs --apply` is the only command that writes into the primary working tree outside `.afu/`, and only after interactive confirmation.

- Node IDs `n*` are stable within one `contract_id`. A new `--approve` mints a new `contract_id` and renumbers. Every file in §6.5 that references nodes carries `contract_id`; mismatch exits 2.
- Symbols are always `path#symbol` in JSON.
- `afu doctor` wraps `entire graph doctor --json`, runs the §7.4 `--assert` block, verifies the demo symbol resolves with ≥ 3 callers at `--profile full`, and prints the model-credential line (§3.3 rule 8). This is the T+0:20 gate as a command.

6.3 Repository layout

```
afu/
├── afu/
│   ├── cli.py         # Typer app                                  (C)
│   ├── graph.py       # subprocess wrapper + parsers               (A)
│   ├── classify.py    # F2: change class × bucket × resolution     (A)
│   ├── plan.py        # graphlib topological sort                  (A)
│   ├── contract.py    # F4 violations, live state, diff cache      (B)
│   ├── shadow.py      # F5 worktree counterfactual                 (B)
│   ├── checkpoint.py  # F6 ledger + staleness          (stretch)  (B)
│   ├── docs.py        # F8 detection — model-free       (stretch)  (B)
│   ├── suggest.py     # F8 draft — lazy import, optional extra     (post-freeze)
│   ├── state.py       # §6.5 readers/writers, contract_id checks   (B)
│   └── render.py      # Rich tree + report                         (C)
├── demo/              # demo repo, planted break, recorded naive patch, doc-map (C)
├── pyproject.toml     # extras: [suggest]
└── README.md
```

`docs.py` consumes the `DiffResult` that `contract.py` already parsed and adds no graph interface function. `suggest.py` is the only module that imports a model SDK; `docs.py` imports it inside the `--suggest` branch only.

6.4 The one interface everything depends on

`graph.py` exposes exactly six functions. Nobody else calls the CLI directly. Write this first, together, in the first twenty minutes.

```
def impact(symbol, depth=2, repo=".") -> ImpactResult
def diff(base, head, repo=".") -> DiffResult          # entities carry change_class + exported flag
def verify(test_cmd, baseline=None, repo=".") -> VerifyResult
def explain(failure_text, repo=".") -> str            # stdin, not argument
def callers(symbol, depth=2, repo=".") -> list[Edge]  # edges --to, fallback neighbors; Edge.id
def capabilities(repo=".") -> Capabilities            # unknown-bucket floor
```

Binding notes.

1. `impact` has no documented JSON flag. Assume text parsing until the T+0:20 gate proves otherwise. Text opens `Index: cache-hit|cache-miss`, then `Impact: <symbol> (file:line)`, then `Blast radius:`, then labelled sections. The `Blast radius:` line is also the source of `--radius` counts.
2. `explain` reads stdin: `subprocess.run(..., input=failure_text)`.
3. `callers()` backs `plan` via `edges --format ndjson --to <symbol> --relation CALLS`, fallback `neighbors --direction in`. `Edge.id` is what makes a node `sound`.
4. `capabilities()` is a real JSON command; it feeds the unknown-bucket floor.

Working tree vs committed tree. `impact`, `search`, `neighbors`, `def`, `explain` read the working tree by default and take `--head`. `snapshot`, `symbols`, `edges` read the committed tree by default and take `--worktree`. Pin the flag explicitly in every call.

6.5 State files — who writes, who reads, how much to trust

Everything under `.afu/`. Three trust levels: **ground truth** (derived from the graph, drives verdicts), **metadata** (bookkeeping; never drives a verdict), **human-owned** (maintained by people; read-only to `afu`).

File	Trust	Written by	Read by	Key fields
`contract.json`	ground truth	`map --approve`	`check`, `shadow`, `docs`, `checkpoint`, `stale`	F1 JSON + `contract_id`, `approved_at`, `open_questions`
`state.json`	ground truth	`check --phase`	`check`, `render`	`contract_id`, per-node `green/red/drift`, `last_phase`
`last-diff.json`	ground truth (cache)	`check`, `docs` (on miss)	`docs`	`base`, `head`, `tree_hash`, `DiffResult`
`shadow.json`	ground truth	`shadow`	`check` (final), `render`	`contract_id`, `naive_broke[]`, `guided_broke[]`, timings
`checkpoint.json`	metadata	`checkpoint write`	`stale`	`contract_id`, evidence hashes, decisions, `open_questions`
`suggestions.json`	metadata	`docs --suggest`	`docs --apply`	`run_id`, `contract_id`, `doc_map_hash`, candidates with `SUGGESTED` drafts, `target_hash`
`docs-state.json`	metadata	`docs --apply`	`docs` (suppression)	applied records with `origin`, signature and section hashes
`doc-map.json`	human-owned	maintainer	`docs`	`entries[]` of `symbol` → `doc_file` [+ `anchor`]

Rules: ground-truth files are written only by the command named and only from graph output. Metadata files are never read by `map`, `plan`, or `check`'s verdict path. `check` refuses a `state.json` or `shadow.json` whose `contract_id` does not match (exit 2). `.afu/` is git-ignored except `doc-map.json`.

---

7. Dependencies

7.1 Required

Dependency	Purpose	Source
`entire` CLI	Plugin host for the graph	github.com/entireio/cli (MIT, Go)
`entire-graph` (MIT)	The code graph	github.com/entireio/entire-graph
Go 1.24+ and a C compiler	To install the above	—
Git 2.5+	Runtime for the graph; `git worktree` for `shadow`	—
Python 3.11+	`graphlib` in stdlib	—
Typer, Rich	CLI and rendering (Rich `Tree` renders the map)	fastapi/typer · Textualize/rich

7.2 Setup sequence

```
go install github.com/entireio/entire-graph/cmd/entire-graph@main
entire plugin install "$(go env GOBIN | grep . || echo "$(go env GOPATH)/bin")/entire-graph" --
force
entire graph init-agents
entire graph doctor --json          # confirm no_egress=true
pip install typer rich              # pip install 'afu[suggest]' adds the model SDK — optional,
post-freeze
```

7.3 Contingency dependencies

If	Then	Source
Symbol resolution comes back thin	`scip-typescript` for compiler-accurate resolution	sourcegraph/scip-typescript
Python repo instead	`scip-python`	sourcegraph/scip-python
Structural search/rewrite needed	`ast-grep`	ast-grep/ast-grep
Nothing parses the target language	git co-change only; every node becomes `guessed` and the map says so	stdlib

7.4 Verified command surface and the T+0:20 gate

Commands used, with published flags: `doctor` (`--json`, `--assert`), `capabilities` (`--json`), `impact` (`--symbol`, `--depth`, `--limit`, `--exclude-tests`, `--file/--line/--kind`, `--head`), `neighbors` (`--symbol`, `--relation`, `--direction`, `--depth`, `--internal-only`), `edges` (`--format ndjson`, `--to`, `--from`, `--relation`, `--worktree`), `diff`/`analyze` (`--base`, `--head`), `commit` (`[REV]`), `verify` (`--test`, `--record-baseline`, `--pre-edit-baseline`, `--setup`), `explain` (stdin), `snapshot` / `snapshot-query`, `index`, `search`. **Two flags decide the build:** `--profile full` (default `fast` under-reports edges and empties `will_break` silently) and `doctor --assert "<cmdline>"` (catches flag drift in seconds).

Run verbatim, in order, before anyone writes code:

```
entire graph doctor --json | grep no_egress
entire graph doctor --assert "impact --symbol useLikedSongs --depth 2 --format json"
entire graph doctor --assert "impact --symbol useLikedSongs --depth 2"
entire graph doctor --assert "edges --format ndjson --to useLikedSongs --relation CALLS"
entire graph doctor --assert "neighbors --symbol useLikedSongs --relation CALLS --direction in"
entire graph doctor --assert "diff --base main --head HEAD"
entire graph capabilities --json | head
entire graph impact --repo . --symbol useLikedSongs --depth 2 --profile full   # ≥ 3 resolved
callers
git worktree add /tmp/afu-shadow-test HEAD && git worktree remove /tmp/afu-shadow-test
```

Line 2 is the fork: clean ⇒ `graph.py` parses JSON; rejected ⇒ lane A writes the text parser. The binary decides.

7.5 Optional model dependency — `docs --suggest` only

One model SDK behind the `[suggest]` extra, imported lazily by `suggest.py`. `afu doctor` does not require it. Its absence degrades exactly one flag on exactly one command. Egress from this call is named in §3.3 rule 8.

---

8. Build plan — 3 hours, 3 engineers

A — Graph. `graph.py`, `classify.py`, `plan.py` · **B — Contract.** `contract.py`, `state.py`, `shadow.py`, then stretch `docs.py`, `checkpoint.py` · **C — Demo.** demo repo, `cli.py`, `render.py`, slides, recording

Time	A	B	C
0:00–0:20	All three: install, §7.4 block, pick and verify demo symbol, `graph index` + `snapshot --format compact-ndjson`. C records the naive patch and writes a two-entry `doc-map.json`.		
0:20–1:00	`impact()` + `capabilities()`; change class × bucket × resolution; `public_exports` list	`diff()` with `change_class` mapping; violations incl. `PHASE_DRIFT`, `UNKNOWN_TOUCHED`; `state.py` writers with `contract_id`; verify baseline	demo repo, plant the break, confirm red; Rich `Tree` skeleton on a hand-written JSON fixture
1:00–1:40	`callers()` via `edges`, topological sort, `order` stamping	`afu check` exit codes, report block, `--phase` state, `last-diff.json` cache	`map` on real `brief` JSON; `--approve` writes contract + `open_questions`
1:40–2:00	**Integration** — one terminal, all three. `map --approve → edit → check --phase → check` end to end.		

Time	A	B	C
2:00–2:20	stretch: `afu stale`	`shadow.py` on the recorded patch → `shadow.json`	slides, the four numbers
2:20–2:40	**Record the demo video. Non-negotiable.**		
2:40–2:45	Freeze, README, submit		
post-freeze	—	`docs.py` detection (≈20 min on cached `DiffResult`), then `suggest.py`	—

Gates.

- **T+0:20** — §7.4 block passes; `impact` returns ≥ 3 resolved callers at `--profile full`. No code before this.
- **T+1:40** — if not integrated, cut `plan` ordering and `--history`; ship `map` (brief only) + `check`. Tree with glyphs plus contract check is the product.
- **T+2:00** — if `shadow` is not running on the recorded patch, the naive-vs-guided number is measured by hand in Beat 1 and `shadow` moves to the roadmap slide.
- **T+2:20** — video recorded regardless.
- **F8 is not on the critical path.** Detection ships only if lane B is idle before T+2:20, which is unlikely; it is honestly listed as post-freeze. The draft layer is post-freeze by design, so the recorded demo's "deterministic and offline" line is true of everything on screen.

---

9. Demo script — 90 seconds

Beat 1 · The pain (20s). Agent changes the hook's return shape. Naive run. Build red, two components broken. Pipe the failure through `entire graph explain` — the graph can describe the wreck but not prevent it. That gap is the product.

Beat 2 · The map (25s). Same task. `afu map --symbol useLikedSongs --radius`. Five files, glyphs, one `○ unknown` sitting exactly where the handler map is, order numbers on every node, frozen signatures and public exports at the bottom. "*The agent has not read a file yet. It already knows the blast zone and where we can't see.*" `--approve`.

Beat 3 · Prevention and the block (30s). Agent executes phase by phase; `afu check --phase 2` shows green. Push the agent out of scope — change `formatDuration`. `afu check` blocks with `FROZEN_SIGNATURE`, exit 1, and the report's last lines read `shadow naive broke 2 · guided broke 0` and `docs 1 candidate · 1 mapped → afu docs`. Measured, not asserted.

Beat 4 · Memory (15s, stretch). Fresh session, 40 node IDs, a few hundred tokens. Two grounding nodes changed; that decision is `unverified`; re-check only those. `open_questions` already contains the registry.ts question — nobody typed it.

Closing slide — four measured numbers.

1. Naive broke 2 components; guided broke 0. Same task, same model, measured by `afu shadow`.
2. 1 unknown region surfaced before the edit, at the file where resolution failed.
3. 9 of 412 tests run, 11 seconds, same failure caught.
4. Fresh session rehydrated in ~340 tokens vs ~61,000 pasting source.

If F8 detection lands, a fifth line: "*N doc-affecting changes flagged, M matched to a doc file.*" Never a fifth beat; the 90 seconds are frozen.

10. Success metrics

Metric	Target	How measured
Broken intermediate states, guided run	0	build after each phase
Blocking violations caught	≥ 1, live	`afu check` exit 1
Map latency	< 3s	wall clock, snapshot-query path
Non-sound nodes with a reason string	100%	JSON schema check
`sound` nodes with ≥ 1 edge ID	100%	JSON schema check
Verdict words in any output	0	grep demo transcript for safe / clean / verified / OK
Naive-vs-guided delta	printed every run	`shadow` line present in report
Duplicate `graph diff` calls in `check` → `docs`	0	subprocess log
`contract_id` mismatches accepted	0	fault-injection test
Tests run vs total	< 5%	verify output
Rehydration cost	< 1,000 tokens	ledger size
Doc candidates without `change_resolution` + `doc_link`	0	JSON schema check
Doc-map hits that changed `change_resolution`	0	grep
Drafted text without `SUGGESTED` label	0	JSON schema check
Doc writes without TTY confirmation in-process	0	code-path audit: one write call, gated on prompt

11. Risks

Risk	L	I	Mitigation
Demo symbol resolves thin	M	Fatal	T+0:20 gate; fallback symbol; scip-typescript rescue
`impact` has no JSON output	M	M	`doctor --assert` decides in five minutes; text parser pre-agreed, lane A owns it

Risk	L	I	Mitigation
Wrong parse profile (`fast` not `full`)	M	H	`--profile full` hardcoded in `graph.py`; T+0:20 caller count catches it
Worktree/committed-tree mismatch	M	M	Every subprocess call pins `--head` or `--worktree`
Map rendering eats the clock	M	H	C builds the Rich tree on a JSON fixture from 0:20; renderer and data land in parallel
`shadow` worktree slow or flaky	M	M	Naive patch recorded at setup; T+2:00 gate drops to hand-measured number
State file from a previous `--approve` applied to a new contract	M	H	`contract_id` in every §6.5 file; mismatch exits 2
Three axes confuse the audience	L	M	One rehearsed sentence: "*class is what changed; bucket is how likely it breaks; glyph is how sure we are it's connected.*"
Agent integration eats the clock	M	H	Past T+1:40, hardcode the proposed diff — the graph layer is what's judged
Model unavailable at demo	L	Fatal	Video at T+2:20; pipeline runs with the model off, tested explicitly
Three modules don't integrate	M	H	Fixed `graph.py` interface written first, by all three together
Doc-map missing for the demo repo	H	L	Zero entries is valid; every candidate reports unmapped, which is the honest output
`--suggest` model call unavailable	M	L	AC9: one candidate degrades to `unavailable`; exit code unchanged; demo shows plain `docs`
`--apply` writes wrong section or without confirmation	L	H	One write call, one candidate per call, `target_hash` check, TTY prompt; no batch path exists to have a bug in

Risk	L	I	Mitigation
Model-drafted text presented as fact	L	H	§3.3 rule 7 label in every path; AC6 is a schema check; `suggestions.json` is metadata no verdict reads
`--suggest` egress undermines the offline story	L	M	§3.3 rule 8: named, off by default, printed by `doctor`; nothing in the demo uses it
Feature creep	H	H	Six commands in the demo. Judges score the demo they saw. Appendix A *Next* stays *Next*

Appendix A — Feature register

#	Feature	Status
1	Pre-edit impact brief (`brief`) with `public_exports`	Shipping (P0)
2	Rendered blast map with resolution glyphs (`map`)	Shipping (P0)
3	Change class × risk bucket × resolution state classification	Shipping (P0)
4	Ordered refactor plan, order stamped on map (`plan`)	Shipping (P0)
5	Map-as-contract (`--approve`), `contract_id`	Shipping (P0)
6	Scope contract check, live `--phase` and final, diff cache (`check`)	Shipping (P0)
7	Blast-radius breakdown (`--radius`)	Shipping (P0, if T+1:40 holds)
8	Counterfactual shadow run (`shadow`) → `shadow.json`	P1, demo-critical
9	Repo memory per node (`--history`)	P1
10	Checkpoint ledger + staleness, auto-seeded `open_questions`	Stretch (P1)
11	Test selection + verify baseline, feeds coverage on map	Stretch (P2)
12	Doc-staleness *detection* (`afu docs` , in `check` final and standalone)	Stretch (P2), realistically post-freeze

#	Feature	Status
13	Model-assisted doc *draft*, accept/edit/reject (`--suggest` / `--apply`)	Post-freeze
14	Suspect ranking on test failure	Next
15	Feature-level rollup ("affects Search, Liked, Player")	Next
16	MCP server exposing `map / approve / check / docs` to any agent (Appendix E is its schema)	Next
17	Ship as a native Entire plugin binary	Next
18	Learned co-change threshold from labelled commit history	Next

Roadmap answer for judges. Suspect ranking, feature rollup so the map reads at Priya's level, an MCP server so Claude Code, Cursor and Codex call `afu map` natively, native plugin packaging (`entire afu ...`), and a learned co-change threshold.

Appendix B — Q&A

"How is this different from a linter?" A linter checks the code you wrote. This shows you the code you're about to touch, tells you how sure it is about every edge, then checks what you wrote against what you declared. When it cannot resolve something it says `unknown` at the file where it gave up, not `clean` at the bottom.

"Isn't `map` just `brief` pretty-printed?" Yes, and that is the point. One query, four views. The tree adds two things the JSON does not: a human can veto before tokens are spent, and `--approve` turns the view into the contract `check` enforces. Picture and rule are the same object, so they cannot drift.

"Why three axes — class, bucket, resolution?" They answer different questions and conflating them is how tools lie. A hop-2 caller is `sound` but only `might_break`. A co-change partner `might_break` but is only `guessed`. A `body` change is `sound` to the graph but `unknown` to the docs. Collapsing them to one score hides exactly what Priya needs.

"What about dynamic dispatch, reflection, DI?" Structurally invisible to a call graph. They go in the `unknown` bucket with an `○` glyph at the file where they live, and git co-change runs as an independent, parser-free layer marked `◐ guessed`.

"Why should we believe your unknown bucket is honest?" Its floor comes from `entire graph capabilities --json` — the graph's own coverage report — plus every call site the parser returns without a resolved target. We read coverage; we do not estimate it. And `sound` requires an edge ID; no code path marks a node sound by heuristic.

"Why should we believe the naive-vs-guided number?" `afu shadow` measures it in a worktree on every run and prints the guided count even when it isn't zero. It is a line in the report, not a slide.

"Where does 60% over 50 commits come from?" A hand-picked threshold, and we say so. It only ever moves a file into `might_break` as `guessed`. It can never promote anything to `sound` or `will_break`. A badly chosen number costs attention, not correctness. Learning it is roadmap item 18.

"Why doesn't `afu docs` diff the docs themselves?" Docs aren't in the graph. We can only tell you code changed in a way *likely* to make a doc wrong; whether it did, and which sentence, is a human read. So every doc line ends in a doc link, not a pass/fail.

"Why is `body` always `unknown` for docs, even on a public export?" A signature staying the same tells you nothing about whether the returned value, thrown error, or default changed — exactly what docs describe. Promoting it would be the confident wrong verdict §3.3 exists to prevent.

"Isn't a doc-map hit a name match, which rule 3 calls `guessed`?" Yes — which is why it lives in its own field, `doc_link: mapped|unmapped`, and never touches `change_resolution`. The change grade comes from the graph; the doc link comes from a human-maintained file. Different provenance, different field.

"Doesn't `--suggest` contradict 'deterministic and offline'?" It is a scoped, named exception. `brief`, `map`, `plan`, `check`, `shadow` never call a model. `--suggest` claims certainty about nothing: labelled `SUGGESTED` in every output, stored only in metadata no verdict reads, written to disk only after a human confirms in that session. It is the honesty principle turned outward — a drafted paragraph is exactly as unverified as a `guessed` node, and we say so with the same discipline. Rule 8 names its egress so nobody discovers it later.

"Why can't `--suggest` batch-apply?" The failure we defend against is a wrong doc edit shipped silently. A batch flag is precisely a way to make that silent.

"Isn't the graph the sponsor's product?" Yes — and we say so first. What we built is the decision layer: the map, the ordering, the contract, the counterfactual, the doc flag, and honesty about what could not be resolved.

"What did you use from the graph that a wrapper wouldn't?" `edges --to` and `neighbors --direction in` for the reverse subgraph; `capabilities` for the coverage floor; `snapshot-query` for sub-3s maps; `commit` dependent counts to gate removal; `explain` on stdin inside the shadow worktree; `doctor --assert` as a build-time contract against the installed binary.

Appendix C — Sponsor repository map

`entire-graph` (substrate; read `AGENTS.md` before `README.md` — it is what `graph.py` codes against). `cli` (plugin host; origin of the checkpoint concept F6 borrows — say so). `entire-search-bench` (the naive-vs-guided harness shape `shadow` borrows). `skills` (packaging pattern for roadmap item 16). `entire-judge` (may read our commit history; commit in real increments). Others not used.

Appendix D — References for §2.2 point 4

To be filled with the two call-graph soundness studies (Java reachable-method recall; JS call-site resolution under precision-tuned tooling) before submission. The numbers carry persuasive weight and must be traceable.

Appendix E — Agent integration contract

Written for the agent (and for whoever builds the agent's classifiers, workflows, and trees next). Everything here is a restatement of §5–§6 in the order the agent encounters it.

E.1 The workflow, as the agent runs it

```
1. Normalize intent   →  symbol (path#symbol) + change_class           (agent's job; see E.4)
2. afu map --symbol S --change C --json                                read nodes[], plan[],
unknown
3. Decide             →  proceed / narrow scope / ask human            (E.3 rules)
4. afu map ... --approve                                               get contract_id
5. For phase in plan:
     edit only nodes[] where order ∈ phase and action == "edit"
     afu check --phase N --json                                         exit 0 → next phase
                                                                       exit 1 → read delta brief,
fix, repeat
6. afu check --json                                                    final report
7. If any candidate in report.docs → surface to human (never --apply yourself)
8. afu checkpoint write                                                before the session ends
```

E.2 Fields the agent reads, and what each licenses

Field	Source	What it licenses
`nodes[].action == "edit"`	contract	The only files the agent may modify
`nodes[].action == "read-only"`	contract	May read; any edit ⇒ `OUT_OF_SCOPE` or `UNKNOWN_TOUCHED`
`nodes[].order`, `plan[].phase`	contract	The only permitted sequence; edits ahead of phase ⇒ `PHASE_DRIFT`
`frozen_signatures[]`	contract	Symbols whose exported signature must not change ⇒ `FROZEN_SIGNATURE`
`public_exports[]`	contract	Symbols whose change will produce a doc candidate; agent should expect the report line
`nodes[].resolution == "guessed"`	contract	Edit is permitted but the agent must run the covering test and state the assumption in its plan
`unknown.*`, `open_questions[]`	contract	Escalate before editing anything in those files
`state.json` per-node `red` + delta brief	check	The specific node and consumers to fix next; not a licence to widen scope
`report.docs.candidates[]`	check final	Information for the human; the agent never applies doc changes

E.3 Decision rules (what the agent must never do)

1. Never parse ANSI. `--json` only.
2. Never edit a file not in `nodes[]` with `action: edit`. If the task requires it, stop and re-run `map` with a wider intent; do not proceed under the old `contract_id`.
3. Never edit a node in a later phase than the current one.
4. Never touch a `○ unknown` file without a human answer to the matching `open_questions[]` entry.
5. Never call `docs --apply`. It is a human command by design.

6. Never treat exit 0 from `check` as "safe." It means "no blocking violation among what the graph could resolve." Read the resolution line.
7. On any `contract_id` mismatch (exit 2), stop; the world moved.

E.4 Agent classifiers we will build

These are not speculative extensions — they are the committed next step, built on top of `afu`'s JSON, never inside it. `afu` stays the deterministic contract layer; the classifiers are the decision layer that consumes it. Each is a separate, testable component with its own input/output contract, not a prompt buried inside `afu` — that separation is what keeps `afu`'s honesty rules (§3.3) enforceable, since nothing upstream of them calls a model.

Classifier	Inputs (all from `afu` JSON)	Output labels	Runs	Dangerous-direction bias
Intent normalizer	Free-text task description; `entire graph search` results, used to validate the guessed symbol	`{symbol: path#symbol, change_class}` — F2's six-value enum	Once, before the first `brief` / `map` call	Misclassifying `signature` as `body` is the dangerous direction — it silently empties `will_break` (F2). Bias toward signature-class whenever the task is ambiguous about return shape, parameters, or removal.
Scope-decision classifier	`radius.{files,packages, public_exports,consumers,tests}`; resolution counts (`sound` / `guessed` / `unknown`); `nodes[].history.red_in_last`	`proceed` | `narrow_scope` | `escalate_to_human`	Once per `map` call, before `--approve`	Any non-empty `unknown` bucket with an unanswered `open_questions[]` entry biases toward `escalate`, never `proceed` — this is what keeps G4/G8 true one layer downstream of `afu`, not just inside it.
Doc-escalation classifier *(post-freeze, pairs with F8)*	`report.docs.candidates[].{change_class, change_resolution, doc_link}`	`notify_now` | `batch_notify` | `ignore` (never `auto_apply`)	Once per `check` final report	`--apply` is a human command by design (E.3 rule 5); this classifier may only route a notification, never call `--apply` itself.

E.5 Other hooks — workflows, trees

- **Phase executor.** A loop over `plan[]` with `check --phase` as the only oracle. The delta brief is the retry prompt.
- **Trees.** The rendered map is the same JSON; a UI, IDE, or agent-side tree view should consume `nodes[]` and `plan[]` and honour the sort rule (unknown → guessed → sound) so risk stays at the top.

- **Stable keys.** Symbols are `path#symbol` everywhere; node IDs are per-`contract_id`; doc candidates are `d*`. Anything persisted across contracts must key on `path#symbol`, never on `n*`.
- **Schema versioning.** Every §6.5 file carries `"schema": 1`. Bump on any field change; readers refuse unknown majors with exit 2.